

International Storm Data Analysis Using Python

Complete VS Code Portfolio Project Guide

SAS-to-Python workflow | pandas | NumPy | Matplotlib | Jupyter | Excel | GitHub

Build locally -> validate -> export -> publish to GitHub -> feature on your website

Project Roadmap

- 1. Project setup in VS Code
- 2. Section 1 - Access Data
- 3. Section 2 - Explore Data
- 4. Section 3 - Prepare Data
- 5. Section 4 - Analyze and Report
- 6. Section 5 - Export Results
- 7. Final notebook checks
- 8. README and repository structure
- 9. GitHub publishing steps
- 10. Website portfolio presentation
- 11. SAS-to-Python translation reference
- 12. Troubleshooting and quality checks

1. Project Setup in VS Code

Use one clean project folder and open the entire folder in VS Code. This keeps file paths stable and makes the project easy to publish to GitHub.

```
storm-data-analysis/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── notebooks/  
│   └── storm_data_analysis.ipynb  
├── output/  
│   ├── tables/  
│   └── figures/  
├── README.md  
├── requirements.txt  
└── .gitignore
```

1.1 Raw input files

Place the original source files in data/raw/. Keep raw source data unchanged.

```
data/raw/  
├── storm.xlsx  
├── storm_detail.sas7bdat  
├── storm_summary.sas7bdat  
└── storm_2017.sas7bdat
```

1.2 Create and activate a virtual environment

```
python -m venv .venv  
.venv\Scripts\activate
```

Install the packages:

```
pip install pandas numpy matplotlib openpyxl jupyter
```

Save the environment:

```
pip freeze > requirements.txt
```

1.3 .gitignore

```
# Python
__pycache__/
*.pyc

# Virtual environments
.venv/
venv/

# Jupyter
.ipynb_checkpoints/

# VS Code
.vscode/

# OS files
.DS_Store
Thumbs.db

# Temporary files
*.tmp
*.log
```

1.4 Notebook title cell

```
# International Storm Data Analysis Using Python

This project demonstrates an end-to-end data analysis workflow using international storm data.

## Workflow
1. Access Data
2. Explore Data
3. Prepare Data
4. Analyze and Report
5. Export Results
```

2. Section 1 - Access Data

This section imports one Excel workbook and three SAS datasets directly into pandas.

2.1 Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

2.2 Project paths

```
PROJECT_DIR = Path.cwd().parent

RAW_DATA_DIR = PROJECT_DIR / "data" / "raw"
PROCESSED_DATA_DIR = PROJECT_DIR / "data" / "processed"
TABLES_DIR = PROJECT_DIR / "output" / "tables"
FIGURES_DIR = PROJECT_DIR / "output" / "figures"

print("Project directory:", PROJECT_DIR)
print("Raw data directory:", RAW_DATA_DIR)
```

2.3 Load Excel data

```
storm_damage = pd.read_excel(
    RAW_DATA_DIR / "storm.xlsx",
    sheet_name="Storm_Damage"
)

storm_damage.head()
```

2.4 Load SAS datasets

```
storm_summary = pd.read_sas(
    RAW_DATA_DIR / "storm_summary.sas7bdat",
    format="sas7bdat",
    encoding="utf-8"
)

storm_2017 = pd.read_sas(
    RAW_DATA_DIR / "storm_2017.sas7bdat",
    format="sas7bdat",
    encoding="utf-8"
)

storm_detail = pd.read_sas(
    RAW_DATA_DIR / "storm_detail.sas7bdat",
    format="sas7bdat",
    encoding="utf-8"
)
```

2.5 Verify the imports

```
print("storm_damage:", storm_damage.shape)
print("storm_summary:", storm_summary.shape)
print("storm_2017:", storm_2017.shape)
print("storm_detail:", storm_detail.shape)

display(storm_damage.head())
display(storm_summary.head())
display(storm_2017.head())
display(storm_detail.head())
```

2.6 Inspect variable names

```
print("storm_damage columns:")
print(storm_damage.columns.tolist())
```

```
print("\nstorm_summary columns:")
print(storm_summary.columns.tolist())

print("\nstorm_2017 columns:")
print(storm_2017.columns.tolist())

print("\nstorm_detail columns:")
print(storm_detail.columns.tolist())
```

3. Section 2 - Explore Data

The exploration phase reproduces PROC FREQ, PROC MEANS, and PROC PRINT using pandas.

3.1 Basin frequency table

```
basin_frequency = (
    storm_summary["Basin"]
    .value_counts()
    .rename_axis("Basin")
    .reset_index(name="Frequency")
)

basin_frequency
```

3.2 Storm type frequency table

```
type_frequency = (
    storm_summary["Type"]
    .value_counts()
    .rename_axis("Type")
    .reset_index(name="Frequency")
)

type_frequency
```

3.3 Summary statistics

```
summary_statistics = storm_summary[
    ["MaxWindMPH", "MinPressure"]
].describe()

summary_statistics
```

A closer PROC MEANS-style version:

```
storm_summary[
    ["MaxWindMPH", "MinPressure"]
].agg(["count", "mean", "std", "min", "max"])
```

3.4 First five imported storm-damage rows

```
storm_damage.head()
```

4. Section 3 - Prepare Data

This is the core transformation section. It translates SAS DATA-step logic into pandas operations.

4.1 Prepare storm_2017

```
storm_2017_clean = (  
    storm_2017  
    .drop(columns=["Location"], errors="ignore")  
    .rename(columns={"Year": "Season"})  
)  
  
storm_2017_clean.head()
```

4.2 Concatenate storm datasets

```
storm_summary2 = pd.concat(  
    [storm_summary, storm_2017_clean],  
    ignore_index=True  
)  
  
storm_summary2.head()
```

4.3 Standardize Basin and create OceanCode

```
storm_summary2["Basin"] = (  
    storm_summary2["Basin"]  
    .str.upper()  
)  
  
storm_summary2["OceanCode"] = (  
    storm_summary2["Basin"]  
    .str[1]  
)
```

4.4 Standardize Season before creating the key

SAS numeric values can arrive in pandas as floats such as 1980.0. Convert Season to a nullable integer before concatenating it with Name.

```
storm_summary2["Season"] = (  
    pd.to_numeric(storm_summary2["Season"], errors="coerce")  
    .astype("Int64")  
)  
  
storm_summary2["Name"] = (  
    storm_summary2["Name"]  
    .astype("string")  
    .str.strip()  
    .str.upper()  
)  
  
storm_summary2["key"] = (  
    storm_summary2["Season"].astype("string")  
    + storm_summary2["Name"]  
)
```

Correct keys look like 1980AGATHA, not 1980.0AGATHA.

4.5 Convert dates and calculate storm duration

```
storm_summary2["StartDate"] = pd.to_datetime(
    storm_summary2["StartDate"]
)

storm_summary2["EndDate"] = pd.to_datetime(
    storm_summary2["EndDate"]
)

storm_summary2["StormLength"] = (
    storm_summary2["EndDate"]
    - storm_summary2["StartDate"]
).dt.days
```

4.6 Create Ocean and BasinName

```
ocean_map = {
    "A": "Atlantic",
    "P": "Pacific",
    "I": "Indian"
}

storm_summary2["Ocean"] = (
    storm_summary2["OceanCode"]
    .map(ocean_map)
)

basin_map = {
    "NA": "North Atlantic",
    "SA": "South Atlantic",
    "WP": "West Pacific",
    "EP": "East Pacific",
    "SP": "South Pacific",
    "NI": "North Indian",
    "SI": "South Indian"
}

storm_summary2["BasinName"] = (
    storm_summary2["Basin"]
    .map(basin_map)
)
```

4.7 Prepare storm_damage

```
storm_damage2 = storm_damage.copy()

storm_damage2["Date"] = pd.to_datetime(
    storm_damage2["Date"]
)

storm_damage2["Name"] = (
    storm_damage2["Event"]
    .str.split()
```

```

        .str[-1]
        .str.upper()
    )

    storm_damage2["Season"] = (
        storm_damage2["Date"]
        .dt.year
        .astype("Int64")
    )

    storm_damage2["Name"] = (
        storm_damage2["Name"]
        .astype("string")
        .str.strip()
        .str.upper()
    )

    storm_damage2["key"] = (
        storm_damage2["Season"].astype("string")
        + storm_damage2["Name"]
    )

    storm_damage2 = storm_damage2.drop(
        columns=["Event", "Date"]
    )

```

4.8 Diagnose key matching before the join

```

storm_damage2.merge(
    storm_summary2,
    on="key",
    how="outer",
    indicator=True
)["_merge"].value_counts()

```

The value under both should be greater than 0. If it is 0, inspect Season data types and Name formatting before continuing.

4.9 Merge the datasets

```

damage_detail = storm_damage2.merge(
    storm_summary2,
    on="key",
    how="inner",
    suffixes=("_damage", "_storm")
)

```

4.10 Keep required columns and sort by Cost

```

damage_detail = (
    damage_detail[
        [
            "Name_damage",
            "Season_damage",
            "BasinName",
            "MaxWindMPH",

```

```

        "MinPressure",
        "StormLength",
        "Cost"
    ]
]
.rename(
    columns={
        "Name_damage": "Name",
        "Season_damage": "Season"
    }
)
.sort_values(
    by="Cost",
    ascending=False
)
)

damage_detail.head(10)

```

4.11 Save prepared datasets

```

storm_summary2.to_csv(
    PROCESSED_DATA_DIR / "storm_summary_clean.csv",
    index=False
)

damage_detail.to_csv(
    PROCESSED_DATA_DIR / "damage_detail.csv",
    index=False
)

```

5. Section 4 - Analyze and Report

5.1 Analysis parameters

```

YEAR = 2016
BASIN = "North Atlantic"

```

5.2 Filter the selected year

```

storms_year = storm_summary2[
    storm_summary2["Season"] == YEAR
].copy()

print(f"Number of storm records in {YEAR}: {len(storms_year)}")

```

5.3 Storm counts by basin

```

basin_counts = (
    storms_year["BasinName"]
    .value_counts()
    .rename_axis("BasinName")
    .reset_index(name="Frequency")
)

```

```
basin_counts
```

5.4 Basin frequency chart

```
ax = basin_counts.plot(
    x="BasinName",
    y="Frequency",
    kind="bar",
    legend=False,
    figsize=(10, 6)
)

ax.set_title(f"Number of Storms by Basin - {YEAR}")
ax.set_xlabel("Basin")
ax.set_ylabel("Number of Storms")

plt.xticks(rotation=45, ha="right")
plt.tight_layout()

plt.savefig(
    FIGURES_DIR / f"storms_by_basin_{YEAR}.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

5.5 Basin by storm type cross-tabulation

```
basin_type_table = pd.crosstab(
    storms_year["BasinName"],
    storms_year["Type"],
    margins=True,
    margins_name="Total"
)

basin_type_table
```

5.6 Wind statistics by storm

```
storm_detail_year = storm_detail[
    storm_detail["Season"] == YEAR
].copy()

wind_stats = (
    storm_detail_year
    .groupby("Name")["Wind"]
    .agg(
        AvgWind="mean",
        MinWind="min",
        MaxWind="max"
    )
    .reset_index()
)
```

```

wind_stats[["AvgWind", "MinWind", "MaxWind"]] = (
    wind_stats[["AvgWind", "MinWind", "MaxWind"]]
    .round(0)
)

wind_stats.head()

```

5.7 Inspect strongest storms

```

wind_stats.sort_values(
    "MaxWind",
    ascending=False
).head(10)

```

5.8 Prepare map data

```

map_data = storm_summary2[
    (storm_summary2["Season"] == YEAR)
    & (storm_summary2["BasinName"] == BASIN)
].copy()

map_data["MapLabel"] = np.where(
    map_data["MaxWindMPH"] >= 100,
    (
        map_data["Name"]
        + "_"
        + map_data["MaxWindMPH"].round(0).astype("Int64").astype(str)
        + " mph"
    ),
    ""
)

map_data = map_data[
    ["Lat", "Lon", "MapLabel", "MaxWindMPH"]
]

map_data.head()

```

5.9 Create and save storm-location bubble plot

```

figure_name = (
    f"{YEAR}_{BASIN.lower().replace(' ', '_')}_storms.png"
)

plt.figure(figsize=(12, 7))

plt.scatter(
    map_data["Lon"],
    map_data["Lat"],
    s=map_data["MaxWindMPH"] * 2,
    alpha=0.6
)

for _, row in map_data.iterrows():
    if row["MapLabel"]:
        plt.text(

```

```

        row["Lon"],
        row["Lat"],
        row["MapLabel"],
        fontsize=8
    )

plt.title(
    f"Tropical Storms in {YEAR} Season\n{BASIN} Basin"
)
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.tight_layout()

plt.savefig(
    FIGURES_DIR / figure_name,
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```

6. Section 5 - Export Results

Use pandas ExcelWriter to reproduce the multi-sheet reporting role of SAS ODS EXCEL.

6.1 Define the report file

```

report_file = TABLES_DIR / f"storm_report_{YEAR}.xlsx"
print(report_file)

```

6.2 Export the workbook

```

with pd.ExcelWriter(
    report_file,
    engine="openpyxl"
) as writer:

    basin_counts.to_excel(
        writer,
        sheet_name=f"{YEAR} Storms by Basin",
        index=False
    )

    basin_type_table.to_excel(
        writer,
        sheet_name=f"{YEAR} Basin by Type"
    )

    wind_stats.to_excel(
        writer,
        sheet_name=f"{YEAR} Wind Statistics",
        index=False
    )

    damage_detail.to_excel(

```

```

        writer,
        sheet_name="Damage Detail",
        index=False
    )

    map_data.to_excel(
        writer,
        sheet_name=f"{YEAR} Basin Map Data",
        index=False
    )

print(f"Report created successfully:\n{report_file}")

```

6.3 Verify the workbook

```

print("Exists:", report_file.exists())
print(f"File size: {report_file.stat().st_size:,} bytes")

excel_file = pd.ExcelFile(report_file)
excel_file.sheet_names

```

6.4 Export individual CSV tables

```

basin_counts.to_csv(
    TABLES_DIR / f"basin_counts_{YEAR}.csv",
    index=False
)

wind_stats.to_csv(
    TABLES_DIR / f"wind_statistics_{YEAR}.csv",
    index=False
)

damage_detail.to_csv(
    TABLES_DIR / "damage_detail.csv",
    index=False
)

```

6.5 Final completion message

```

print("Storm analysis completed successfully.")
print(f"Analysis year: {YEAR}")
print(f"Selected basin: {BASIN}")
print(f"Excel report: {report_file}")
print(f"Figures saved to: {FIGURES_DIR}")
print(f"Tables saved to: {TABLES_DIR}")

```

7. Final Notebook Checks

- In VS Code, select Run All and confirm every cell executes from top to bottom.
- Confirm all input paths resolve correctly from the notebooks folder.
- Confirm the merge diagnostic reports matched records under both.
- Confirm output PNG files are created in output/figures/.
- Confirm storm_report_2016.xlsx is created in output/tables/.

- Open the Excel workbook and verify all worksheet names and tables.
- Remove exploratory or duplicate cells that are no longer needed.
- Keep Markdown explanations concise so a recruiter can follow the workflow quickly.

8. README.md - Portfolio-Ready Version

```
# International Storm Data Analysis Using Python

## Project Overview
This project demonstrates an end-to-end data analysis workflow using Python and international storm data. The project translates a traditional SAS programming workflow into Python using pandas, NumPy, Matplotlib, and Jupyter.

## Workflow
1. Access Data
2. Explore Data
3. Prepare Data
4. Analyze and Report
5. Export Results

## Technologies Used
- Python
- pandas
- NumPy
- Matplotlib
- Jupyter Notebook
- openpyxl
- SAS datasets (.sas7bdat)
- Excel

## Skills Demonstrated
- Python data analysis
- Data cleaning and transformation
- Data integration and merging
- Exploratory data analysis
- Statistical summaries
- Data visualization
- Report automation
- SAS-to-Python translation
```

9. Publish to GitHub from VS Code

9.1 Initialize Git

```
git init
git status
```

9.2 Stage and commit

```
git add .
git status
git commit -m "Initial storm data analysis project"
```

9.3 Create the GitHub repository

- On GitHub, create a new repository named storm-data-analysis.
- Make it Public for a portfolio project.
- Do not create another README or .gitignore on GitHub because those files already exist locally.

9.4 Connect and push

```
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/storm-data-analysis.git
git push -u origin main
```

9.5 Future updates

```
git add .
git commit -m "Update analysis and documentation"
git push
```

9.6 Detailed GitHub Add and Publish Process

Use these commands in the VS Code terminal after the notebook, README, requirements.txt, and .gitignore are complete. Run them from the root storm-data-analysis folder.

A. Confirm Git is installed

```
git --version
```

If a Git version is displayed, continue. If the command is not recognized, install Git for Windows and restart VS Code.

B. Confirm you are in the correct project folder

```
Get-Location
```

The path should end in storm-data-analysis. You can also run `dir` to confirm that README.md, requirements.txt, notebooks, data, and output are visible.

C. Initialize the local Git repository

```
git init
git status
```

`git init` creates the hidden `.git` folder. `git status` shows which project files are currently untracked or modified.

D. Stage the project with `git add`

```
git add .
git status
```

`git add .` stages all non-ignored files in the current project and its subfolders. Files listed under "Changes to be committed" are ready for the next commit. Your `.gitignore` should prevent local environment files such as `.venv` and `.ipynb_checkpoints` from being staged.

For a single file, use a targeted command such as:

```
git add README.md
```

E. Create the first commit

```
git commit -m "Initial storm data analysis project"
```

A commit records the staged version of the project in local Git history.

F. Create the repository on GitHub

- Sign in to GitHub and choose New repository.
- Repository name: storm-data-analysis.
- Choose Public if this is a portfolio project.
- Do not initialize the GitHub repository with another README, .gitignore, or license when those files already exist locally.
- Create the repository and copy the HTTPS repository URL.

G. Connect the local repository to GitHub

```
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/storm-data-analysis.git
git remote -v
```

Replace YOUR-USERNAME with your actual GitHub username. `git remote -v` lets you verify that origin points to the correct GitHub repository.

H. Push the project to GitHub

```
git push -u origin main
```

The first push may open a GitHub sign-in or authorization window. After it succeeds, refresh the GitHub repository page and confirm the files are visible.

I. Verify the published repository

- README.md renders on the repository home page.
- notebooks/storm_data_analysis.ipynb opens and renders correctly.
- output/figures contains the portfolio charts you intend to show.
- No .venv, passwords, API keys, personal files, or unnecessary temporary files were uploaded.
- The raw data files are appropriate and permitted to publish.

J. Publish future changes

```
git status
git add .
git commit -m "Update storm analysis"
git push
```

This is the normal update cycle after the first publication: review changes, stage them, commit them, and push them to GitHub.

K. Useful checks when something goes wrong

```
git status
git remote -v
git log --oneline -5
```

- If `git add .` appears to do nothing, run `git status` and confirm the file is not excluded by `.gitignore`.
- If origin already exists, inspect it with `git remote -v` before changing it.

- If a push is rejected because the GitHub repository already contains unrelated files, do not force-push automatically; reconcile the histories first.
- If Git asks for your identity before committing, configure your Git user.name and user.email with the values you want attached to commits.

10. Add the Project to Your Website

GitHub should contain the full technical project. Your website should present a concise, polished summary and link to GitHub.

International Storm Data Analysis

End-to-end Python data analysis project using international storm data, including data cleaning, exploratory analysis, statistical summaries, visualization, and automated result exports.

Python | pandas | NumPy | Matplotlib | Jupyter

[View Project on GitHub]

- Use one strong chart from output/figures/.
- Include a 2-3 sentence project summary.
- List the main technologies.
- Add a direct View on GitHub button.
- Optionally mention that the project translates an existing SAS workflow into Python.

11. SAS-to-Python Translation Reference

SAS	Python / pandas
LIBNAME	pathlib.Path / directory variables
PROC IMPORT	pd.read_excel()
SAS7BDAT dataset	pd.read_sas()
PROC FREQ	value_counts() / pd.crosstab()
PROC MEANS	describe() / groupby().agg()
PROC PRINT	head() / display()
DATA step	pandas transformations
SET	pd.concat()
DROP	.drop()
RENAME	.rename()
UPCASE()	.str.upper()
SUBSTR()	.str[]
SCAN()	.str.split()
CATS()	string concatenation
YEAR()	.dt.year
IF / ELSE	map() / np.where()
PROC SQL join	.merge()
WHERE	Boolean filtering
CLASS	.groupby()
ORDER BY	.sort_values()

Macro variables	Python variables/constants
ODS EXCEL	pd.ExcelWriter()
PROC SGMAP	Matplotlib / Plotly / GeoPandas / Folium

12. Troubleshooting and Quality Checks

12.1 Merge produces zero matches

Typical symptom: the merge diagnostic shows both = 0. The most common reason in this project is that Season was converted to strings from floating-point values such as 1980.0, producing keys such as 1980.0AGATHA.

```
storm_summary2["Season"] = pd.to_numeric(
    storm_summary2["Season"], errors="coerce"
).astype("Int64")

storm_damage2["Season"] = pd.to_numeric(
    storm_damage2["Season"], errors="coerce"
).astype("Int64")
```

Then recreate the standardized Name values and keys in both datasets.

12.2 Path errors

- Open the entire storm-data-analysis folder in VS Code, not only the notebook file.
- Run the notebook from notebooks/storm_data_analysis.ipynb.
- Print PROJECT_DIR and RAW_DATA_DIR if a file cannot be found.
- Confirm filenames and extensions exactly match what is in data/raw/.

12.3 Encoding errors when reading SAS files

If utf-8 fails for a particular SAS file, try the file encoding used by the source data, such as latin1. Use one encoding consistently only when it correctly reads the file.

```
pd.read_sas(
    RAW_DATA_DIR / "storm_summary.sas7bdat",
    format="sas7bdat",
    encoding="latin1"
)
```

12.4 Portfolio quality checklist

- Notebook runs from start to finish without manual intervention.
- Code is organized into the five analysis sections.
- Raw data is separated from processed data.
- Outputs are written to dedicated tables and figures folders.
- Charts include clear titles and axis labels.
- README explains the business/analysis workflow rather than only listing code.
- No passwords, API keys, private data, or irrelevant temporary files are committed.
- GitHub notebook renders correctly.
- Website links directly to the GitHub repository.

13. Final Project Outcome

When complete, this project demonstrates a full analytics workflow: importing Excel and SAS data, exploring variables, preparing and integrating datasets, producing descriptive analyses and visualizations, exporting reports, and publishing a reproducible repository. It also provides a concrete example of translating common SAS programming patterns into modern Python/pandas workflows.

```
storm-data-analysis/
├── data/
│   ├── raw/
│   └── processed/
│       ├── storm_summary_clean.csv
│       └── damage_detail.csv
├── notebooks/
│   └── storm_data_analysis.ipynb
├── output/
│   ├── tables/
│   │   ├── storm_report_2016.xlsx
│   │   ├── basin_counts_2016.csv
│   │   ├── wind_statistics_2016.csv
│   │   └── damage_detail.csv
│   └── figures/
│       ├── storms_by_basin_2016.png
│       └── 2016_north_atlantic_storms.png
├── README.md
├── requirements.txt
└── .gitignore
```

14. Final Update - Recommended Production Workflow

Use this final sequence when preparing the project for your portfolio:

1. Run the notebook from top to bottom with Run All in VS Code.
2. Confirm Season is stored as an integer before creating string keys.
3. Confirm the merge diagnostic reports matched rows under both.
4. For the final polished notebook, prefer a direct merge on Season and Name when both columns are standardized.
5. Verify all CSV, Excel, and PNG outputs are created in the expected folders.
6. Review README.md, requirements.txt, and .gitignore before publishing.
7. Commit and push the project to GitHub.
8. Open the GitHub notebook and README to confirm they render correctly.
9. Add the GitHub project link and one strong chart to your portfolio website.

14.1 Preferred final merge

The SAS-style concatenated key is useful for learning and validation. For the final Python portfolio version, a direct multi-column merge is cleaner and avoids formatting problems caused by concatenated keys.

```
storm_damage2["Season"] = pd.to_numeric(
    storm_damage2["Season"], errors="coerce"
).astype("Int64")
```

```
storm_summary2["Season"] = pd.to_numeric(
    storm_summary2["Season"], errors="coerce"
).astype("Int64")
```

```
storm_damage2["Name"] = storm_damage2["Name"].str.strip().str.upper()
storm_summary2["Name"] = storm_summary2["Name"].str.strip().str.upper()
```

```
damage_detail = storm_damage2.merge(
    storm_summary2,
    on=["Season", "Name"],
    how="inner",
    suffixes=("_damage", "_storm")
)
```

14.2 GitHub first-publish command sequence

```
git status
git add .
git commit -m "Initial storm data analysis project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/storm-data-analysis.git
git push -u origin main
```

Before the first push, create an empty public GitHub repository named storm-data-analysis. Do not initialize it with another README or .gitignore if those files already exist locally.

14.3 GitHub update cycle after publication

```
git status  
git add .  
git commit -m "Update storm analysis and documentation"  
git push
```

Use this short cycle every time you change the notebook, README, figures, or other project files after the repository has already been published.

14.4 Final portfolio publishing checklist

- Notebook runs without errors from the first cell to the last.
- README clearly explains the objective, workflow, technologies, outputs, and SAS-to-Python translation.
- No virtual environment, credentials, private files, or temporary files are tracked by Git.
- Raw data is permitted to be published.
- Figures and tables are generated from the notebook rather than edited manually.
- GitHub repository is public and the notebook renders correctly.
- Website project card links directly to the GitHub repository.